Programming Assignment 4

Filename: `pa4.c`

You must include the following commented lines at the beginning of your code to declare authorship. Submissions that do not include these lines will receive a 10% deduction.

```
/* COP 3502C PA4
This program is written by: Your Full Name */
```

In this activity, you will practice implementing sorting algorithms in C, focusing on the following:

- Implementing and applying the Merge Sort and Quick Sort algorithms
- Sorting records using multiple keys with a secondary tie-breaking rule
- Working with structures and pointers to store and manipulate structured data
- Designing and using a `compareTo`-style comparison function to support flexible sorting

Background

A local animal shelter has collected detailed information about all of its cats, including how cute, fluffy, agile, friendly, smart, and lazy each one is. Potential adopters often want to view cats ranked by a specific trait to help them find their ideal pet. For example, one person might want to see the fluffiest cats first, while another might be interested in the smartest or friendliest cats.

Your task in this assignment is to build the ranking system for the shelter. Given information about many cats and their trait scores, your program will sort the cats according to a selected trait and produce an official *Cat Rank List*. The list will show the cats ordered from highest to lowest score for the chosen trait.

Efficiency is important because the shelter may eventually store information about thousands of cats. To handle this efficiently, you will implement two different efficient sorting algorithms, merge sort and quicksort, as learned in class.

Solution Design

This assignment requires you to implement two sorting algorithms: Merge Sort and Quick Sort. For both algorithms, when the size of a subarray becomes less than or equal to a threshold value of 30, the algorithm must switch to Insertion Sort, a quadratic-time sorting algorithm. This hybrid approach improves efficiency because simple algorithms often perform better on small arrays.

Each sorting algorithm must be implemented using a wrapper function that initiates the sorting process. The wrapper function will call the actual recursive sorting function that performs the divide-and-conquer steps. The required function prototypes are provided later in this document.

You must also implement a comparison function that determines the ordering between two cats. This function should compare the cats using the selected trait and break ties alphabetically using the cat's name. Both sorting algorithms must use this comparison function when determining the ordering of elements.

Memory for the cat structures and their names must be dynamically allocated. The array of cat pointers must be allocated based on the value of n. In addition, each cat name must be dynamically allocated using exactly the amount of memory required for the string. You must use the same dynamic string allocation strategy that was discussed in class. Any other approach may result in no credit.

Required Data Structures

You must use the following structure in this assignment. You are allowed to declare more structures if needed. However, you are not allowed to change the required structure.

```
typedef struct Cat_s {
    char *name;                 // Cat's name (unique)
    int scores[NUM_TRAITS];
} Cat;
```

Global Constant Variable

The following must be declared in your code:

```
#define NUM_TRAITS 7
#define TOTAL_IDX 6
#define MAX_LEN 13
#define BASE_CASE_SIZE 30

const char TRAITS[NUM_TRAITS][MAX_LEN] = {
    "Cuteness", "Fluffiness", "Agility", "Friendliness",
    "Intelligence", "Laziness", "Total"
};
```

Constant Descriptions

- `NUM_TRAITS` – the total number of possible sorting keys (six individual traits plus one for the total score).
- `TOTAL_IDX` – the index in the scores array that corresponds to the total score of a cat.
- `MAX_LEN` – the maximum length of all strings used in the program, including both cat names and trait names (including the null terminator).
- `BASE_CASE_SIZE` – the threshold size at which the recursive sorting algorithms must switch to insertion sort. Although insertion sort has a worst-case running time of $O(n^2)$, it performs very efficiently on small arrays and is commonly used as a base case in hybrid sorting algorithms.

Starter Code

Your task is to complete the given C program. You can access the starter code here.

Required Functions

You must implement and use the following functions with the specified prototypes in your solution. You are free to define additional functions as needed (and are recommended to do so).

Important: The 5 required functions must be implemented exactly as specified. Therefore, their prototypes must not be modified.

1. Your `myMain()` function must store an array of pointers to `Cat` structures (type `Cat **`). Each index of this array must contain a pointer to a single `Cat` structure.
2. You must implement the following comparison function:

```
// Returns a negative integer if the cat pointed to by ptrC1
// comes before the cat pointed to by ptrC2 in the rank list
// when sorted by the trait indicated by key.
// Ties must be broken alphabetically by the cat's name.
// Returns 0 if the two cats are identical.
// Returns a positive integer if ptrC2 should come before ptrC1.
int compareTo(Cat *ptrC1, Cat *ptrC2, int key);
```

3. For Merge Sort, your wrapper function must have the following prototype:

```
// Merge sorts the array list of size n according to the trait
// indicated by key.
void mergeSort(Cat **list, int n, int key);
```

4. The recursive Merge Sort function must have the following prototype:

```
// Performs merge sort on list[low..high] according to the
// trait indicated by key.
void mergeSortRec(Cat **list, int low, int high, int key);
```

5. For Quick Sort, your wrapper function must have the following prototype:

```
// Quick sorts the array list of size n according to the trait
// indicated by key.
void quickSort(Cat **list, int n, int key);
```

6. The recursive Quick Sort function must have the following prototype:

```
// Performs quick sort on list[low..high] according to the
// trait indicated by key.
void quickSortRec(Cat **list, int low, int high, int key);
```

7. For your Quick Sort implementation, you must select the pivot randomly during each partition step. Any element within the current subarray may be selected as the pivot. Random pivot selection helps reduce the likelihood of encountering the worst-case running time on already sorted or nearly sorted input.
8. For both Merge Sort and Quick Sort, any subarray of size 30 or smaller must be handled using Insertion Sort. This serves as the base case of the recursive algorithm.

Input

The program reads input from a file named `scores.txt`. The first line contains a single positive integer n $(1 \leq n \leq 10^5)$ representing the number of cats. Each of the next n lines contains information about one cat using the following format:

```
name cuteScore fluffScore agileScore friendlyScore smartScore lazyScore
```

where:

- `name` – the name of the cat. Each name is unique and consists of 1 to 12 lowercase letters.
- `cuteScore` – the cat's cuteness score
- `fluffScore` – the cat's fluffiness score
- `agileScore` – the cat's agility score
- `friendlyScore` – the cat's friendliness score
- `smartScore` – the cat's intelligence score
- `lazyScore` – the cat's laziness score

Each score is an integer between 0 and 1000 (inclusive).

The final line of input contains two integers:

- s – the sorting algorithm to use:
  - $1 =$ Merge Sort
  - $2 =$ Quick Sort
- g $(0 \leq g \leq 6)$ – the trait used to generate the ranking

The value of g corresponds to the following traits:

g	Trait
0	Cuteness
1	Fluffiness
2	Agility
3	Friendliness
4	Intelligence
5	Laziness
6	Total score across all traits

Important Note

You may assume that all input strings contain no whitespace.

Output

The first line of output must have the format:

```
<Trait> Rank List
```

where `<Trait>` is one of the following constant strings stored in the global array `TRAITS`. The index into this array corresponds to the input value g.

The next n lines must display the ranked list of cats sorted by the selected trait from highest score to lowest score. If two cats have the same score for the selected trait, the tie must be broken using the cat's name in alphabetical order.

Each output line must follow the format:

```
x. name score
```

where:

- x – the 1-based rank
- `name` – the cat's name
- `score` – the score for the selected trait (or total score)

The cat's name must be printed using a field width of 15 characters and must be left-justified. Use the following statement:

```
printf("%d. %-15s %d\n", rank, name, score);
```

Sample Input `scores.txt`

```
5
whiskers 90 80 70 95 85 60
mittens 85 95 60 80 75 70
shadow 70 60 95 85 90 50
luna 95 85 75 90 80 65
oreo 60 70 80 60 65 95
1 0
```

Sample Output

```
Cuteness Rank List
1. luna            95
2. whiskers        90
3. mittens         85
4. shadow          70
5. oreo            60
```

Additional Implementation Requirements

- All sorting algorithms must be implemented exactly as presented in lecture. Any implementation that significantly deviates from the approach taught in class will be considered an academic integrity violation and may result in a penalty.
- You are not allowed to use any built-in sorting functions. Using such functions will result in a score of 0 for the assignment.
- Do not declare any global variables other than those explicitly specified in the assignment instructions.
- When reading strings, you must first store the input in a static `char` array. Then allocate the exact amount of memory required for the string and copy it using the `strcpy()` function, following the method demonstrated in class.
- All dynamically allocated memory must be properly freed in order to receive full credit.
- Your program must terminate with no memory leaks.

Clarifications

For questions or clarifications about the program requirements, please post them in the corresponding discussion thread on Webcourses. If additional clarification is needed, you are welcome to visit the TAs or the instructor during office hours. Requests for help with debugging should be brought to office hours rather than sent by email.

Tentative Rubric (Subject to Change)

A set of test cases will be used to evaluate whether your program produces the expected output. Each failed test case will result in a grade deduction per the rubric below. In addition to the sample test cases, we will use additional (hidden) test cases during grading. Therefore, passing the sample test cases does not guarantee full credit. You are strongly encouraged to thoroughly test your code against a variety of inputs. Because some test cases are hidden, your final score will not be visible until the submission has been fully graded.

Important: Any attempt to hard-code the expected output or otherwise circumvent the grading process constitutes a violation of academic integrity and will be handled in accordance with university policy.

If it is later discovered that a submission violates the course AI policy or any academic integrity rules, the assignment may be re-evaluated even after grades have already been posted on Webcourses. In such cases, previously awarded scores may be retracted and the submission may be regraded according to the course policies. Additional academic integrity actions may also be taken if warranted.

- If your code does not compile on Gradescope, you will receive a score of 0.
- If you modify, omit, or fail to use the required data structures, you may receive a score of 0.
- Failure to use dynamic memory allocation for storing data will result in a score of 0.
- Correct implementation of all functions and specifications: 30%.
- Properly freeing all dynamically allocated memory with zero memory leaks: 10%.
- Passing all test cases with exact output matching: 60%.